

LAB 6 — Despliegue con Docker

MLOps-Capstone-Project

Asignatura: MLOps & AI BI | Sesión 6

Alumna: Claudia Román García

Fecha: Marzo 2026

Parte 1: Ejercicios Prácticos

Ejercicio 1 — Backend en solitario

Se construyó y arrancó el contenedor del backend con los siguientes comandos:

```
docker build -t iris-backend .
docker run -d -p 8000:8000 --name iris-backend iris-backend
```

Verificación GET /health

```
curl http://localhost:8000/health
```

Respuesta obtenida:

```
{"status": "ok", "model_loaded": true, "model_version": "v1.0-base", "uptime_seconds": 6.2}
```

El endpoint confirma que el modelo está cargado en memoria y listo para recibir predicciones.

Verificación POST /predict

```
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4, "petal_width": 0.2}'
```

Respuesta obtenida:

```
{"prediction": 0, "species": "setosa", "confidence": 1.0}
```

El backend devuelve correctamente la predicción de especie con confianza del 100%.

Ejercicio 2 — Frontend en solitario

Se construyó y arrancó el contenedor del frontend apuntando al backend del Ejercicio 1:

```
docker build -t iris-frontend .
docker run -d -p 7860:7860 --name iris-frontend \
-e BACKEND_URL=http://host.docker.internal:8000 iris-frontend
```

Se accedió a <http://localhost:7860> y se comprobó que la interfaz cargaba correctamente:

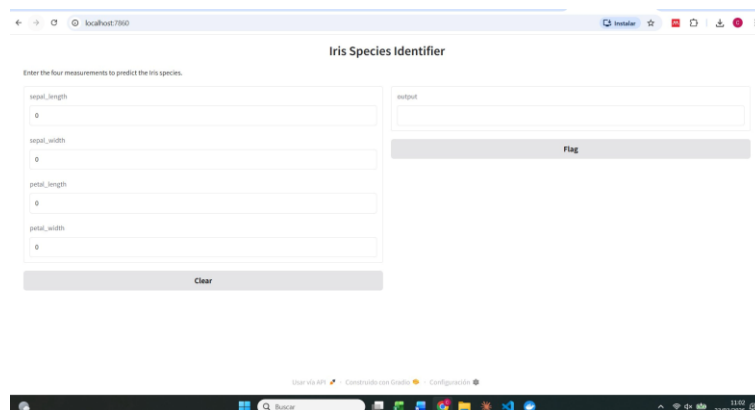


Figura 1 — Interfaz Gradio accesible en <http://localhost:7860>

Se realizó una predicción con los valores: sepal_length=5.1, sepal_width=3.5, petal_length=1.4, petal_width=0.2

Iris Species Identifier

Enter the four measurements to predict the Iris species.

sepal_length
5.1

sepal_width
3.5

petal_length
1.4

petal_width
0.2

Clear

output
setosa (confidence: 100.0%)

Flag

Usar vía API · Construido con Gradio · Configuración

Figura 2 — Predicción: setosa (confidence: 100.0%)

La interfaz Gradio se comunica correctamente con el backend y muestra la predicción al usuario.

Ejercicio 3 — Stack completo con Docker Compose

Se detuvo los contenedores individuales y se arrancó el stack completo con un único comando:

```
docker compose up --build -d
```

Estado de los contenedores verificado con docker compose ps:

NAME	IMAGE	STATUS	PORTS
iris-backend	mlops-crg-backend	Up (healthy)	0.0.0.0:8000->8000/tcp
iris-frontend	mlops-crg-frontend	Up	0.0.0.0:7860->7860/tcp

Predicción end-to-end realizada desde el frontend con los valores: sepal_length=6.3, sepal_width=2.8, petal_length=5.1, petal_width=1.5:

Iris Species Identifier

Enter the four measurements to predict the Iris species.

sepal_length
6.3

sepal_width
2.8

petal_length
5.1

petal_width
1.5

output
virginica (confidence: 100.0%)

Flag

Clear

Usar vía API · Construido con Gradio · Configuración

Figura 3 — Predicción desde el stack completo: virginica (confidence: 100.0%)

El frontend se comunica con el backend usando el nombre de servicio interno backend (<http://backend:8000>), confirmando que la red interna de Docker Compose funciona correctamente.

Ejercicio 4 — Verificación automática end-to-end

Se ejecutó el script de verificación automática:

```
chmod +x verify_stack.sh
./verify_stack.sh
```

Salida obtenida:

```
=====
Verificación del Stack - LAB 6
Backend : http://localhost:8000
Frontend: http://localhost:7860
=====
[OK]    Backend listo en 0s - modelo: v1.0-base
[OK]    Predicción recibida → clase=0, especie=setosa
        Payload enviado:    {"sepal_length": 5.1, ...}
        Respuesta completa: {"prediction":0,"species":"setosa","confidence":1.0}
[OK]    Frontend disponible (HTTP 200).
=====
STACK OK
=====
```

El script confirma que el backend responde correctamente, la predicción es válida y el frontend está disponible.

Parte 2: Preguntas de Reflexión

Pregunta 1 — ¿Por qué es necesario `--host 0.0.0.0`?

Cuando arrancas `uvicorn` sin especificar el `host`, o con `--host 127.0.0.1`, el servidor solo acepta conexiones que vengan desde dentro del propio contenedor. Es como si cerraras la puerta por dentro: el proceso está funcionando, pero nadie de fuera puede llamar.

El problema es que cuando usas Docker y mapeas el puerto con `-p 8000:8000`, las peticiones llegan al contenedor desde fuera, no desde dentro. Si la puerta está cerrada por dentro, esas peticiones simplemente no llegan aunque el contenedor esté corriendo.

Al poner `--host 0.0.0.0` le dices al servidor que acepte conexiones desde cualquier sitio, no solo desde dentro. Así el mapeo de puertos funciona y puedes acceder a la API desde tu navegador o desde el contenedor del frontend.

Pregunta 2 — ¿Por qué usar `'backend'` en vez de `'localhost'`?

Cada contenedor en Docker vive en su propio mundo aislado. Cuando desde el frontend usas `localhost`, estás diciendo 'yo mismo', es decir, el propio contenedor del frontend. No tiene nada que ver con el contenedor del backend, aunque los dos estén corriendo en la misma máquina.

Docker Compose soluciona esto de forma automática: cuando defines varios servicios en el mismo fichero `docker-compose.yml`, Docker crea una red privada compartida entre ellos y asigna a cada servicio su propio nombre como si fuera una dirección. Así, el nombre `backend` funciona como si fuera una dirección web, y Docker se encarga de redirigir la conexión al contenedor correcto.

Por eso en la variable de entorno del frontend usamos `BACKEND_URL=http://backend:8000`. Es mucho más fiable que usar una dirección IP, que además puede cambiar cada vez que se reinician los contenedores.

Pregunta 3 — Caché de capas de Docker

Cuando Docker construye una imagen, lo hace paso a paso siguiendo las instrucciones del `Dockerfile`. Cada instrucción crea una capa, y Docker funciona de forma de que: si una capa no ha cambiado desde la última vez que se construyó, la reutiliza directamente sin volver a ejecutarla. Esto ahorra mucho tiempo.

El problema viene cuando el orden está mal. Si primero copiamos todo el código y luego instalamos las dependencias, cada vez que cambiamos aunque sea una línea de código Docker tiene que volver a instalar todas las librerías desde cero, lo que puede tardar varios minutos.

Con el orden correcto, primero copiamos solo el fichero de dependencias, las instalamos, y después copiamos el código. Así, si solo cambiamos una línea de código, Docker reutiliza la capa de instalación ya guardada y solo recopia el código nuevo. La construcción pasa de tardar minutos a tardar segundos.

En resumen: lo que cambia poco (las dependencias) va primero, y lo que cambia con frecuencia (el código) va al final.

Pregunta 4 — Gestión de secretos

Cuando una aplicación necesita usar un token privado como el de Hugging Face, hay varias formas de pasárselo sin escribirlo directamente en el código:

Variable de entorno del terminal: escribes el token directamente en la ventana de comandos antes de arrancar Docker. Es la opción más rápida pero menos segura, porque el token queda visible en el historial de comandos del ordenador.

Fichero `.env` (el que usamos en este lab): creas un fichero de texto con el token y lo añades al `.gitignore` para que no se suba a GitHub. Es cómodo para trabajar en local, pero el riesgo es que si en algún momento el `.gitignore` falla o te olvidas de añadirlo, el token puede acabar publicado en el repositorio sin querer. Si eso ocurre hay que revocarlo y generar uno nuevo inmediatamente.

Parte 3: Ejercicio de Programación

Health Check y Arranque Ordenado

El problema

Docker Compose puede asegurarse de que el backend arranque antes que el frontend, pero no de que esté realmente listo para responder. El backend necesita descargar el modelo de Hugging Face Hub al iniciarse, y eso puede tardar varios segundos. Si el frontend arranca durante ese tiempo e intenta hacer una predicción, recibirá un error.

Análisis de los tres escenarios

Escenario 1 — modelo pequeño cargado en menos de 1 segundo:

El problema apenas se nota porque el modelo ya está en disco y se carga casi al instante. En este caso un arranque simple funcionaría, aunque no es la mejor práctica.

Escenario 2 — modelo descargado desde Hugging Face Hub al arrancar:

Este es nuestro caso. La descarga puede tardar entre 5 y 30 segundos. Si el frontend arranca antes de que termine, las primeras peticiones fallarán. Aquí es imprescindible que el frontend espere a que el backend confirme que está listo.

Escenario 3 — modelo grande que requiere tiempo de preparación:

El arranque puede tardar minutos. En este caso hay que dar al contenedor suficiente tiempo de gracia antes de empezar a comprobar si está listo, para no marcarlo como fallido antes de que termine de cargar.

Diseño de la solución

La solución tiene tres partes que trabajan juntas:

En el Dockerfile del backend, añadimos un health check que pregunta al servidor cada 10 segundos si está listo, dándole 30 segundos de margen al principio:

```
HEALTHCHECK --interval=10s --timeout=5s --start-period=30s --retries=3  
CMD curl -f http://localhost:8000/health || exit 1
```

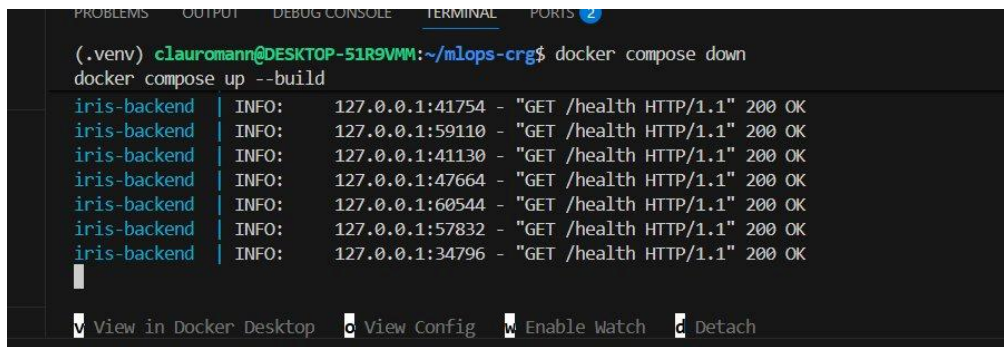
En el código del backend, el endpoint /health responde de forma inteligente: devuelve OK solo cuando el modelo está cargado en memoria, y un error mientras todavía está cargando.

En el docker-compose.yml, el frontend espera a que el backend supere el health check antes de arrancar:

```
depends_on:  
  backend:  
    condition: service_healthy
```

Evidencia del arranque ordenado

Los logs en tiempo real muestran claramente la secuencia de arranque. Primero el backend descarga el modelo y pasa el health check, y solo entonces arranca el frontend:



```
(.venv) clauromann@DESKTOP-51R9VMM:~/mlops-crg$ docker compose down
docker compose up --build
iris-backend | INFO: 127.0.0.1:41754 - "GET /health HTTP/1.1" 200 OK
iris-backend | INFO: 127.0.0.1:59110 - "GET /health HTTP/1.1" 200 OK
iris-backend | INFO: 127.0.0.1:41130 - "GET /health HTTP/1.1" 200 OK
iris-backend | INFO: 127.0.0.1:47664 - "GET /health HTTP/1.1" 200 OK
iris-backend | INFO: 127.0.0.1:60544 - "GET /health HTTP/1.1" 200 OK
iris-backend | INFO: 127.0.0.1:57832 - "GET /health HTTP/1.1" 200 OK
iris-backend | INFO: 127.0.0.1:34796 - "GET /health HTTP/1.1" 200 OK
View in Docker Desktop View Config Enable Watch Detach
```

Figura 4 — Logs del arranque: el frontend espera a que el backend esté healthy

La secuencia completa es:

- El backend inicia y comienza a descargar el modelo en segundo plano.
- Docker lanza el health check cada 10 segundos. Durante los primeros 30 segundos los fallos no cuentan.
- Cuando el modelo termina de cargarse, /health devuelve HTTP 200 y Docker marca el backend como healthy.
- Solo entonces arranca el frontend, garantizando que la primera llamada al backend siempre encontrará el modelo disponible.

El resultado final se puede verificar en el docker compose ps, donde el backend aparece como Up (healthy) y el frontend como Up, confirmando que el arranque ordenado funcionó correctamente.